1. Moveable Shapes Simulation

• Define an interface Movable with methods: moveUp(), moveDown(), moveLeft(), moveRight().

• Implement classes:

- **o MovablePoint(x, y, xSpeed, ySpeed) implements Movable**
- **o MovableCircle(radius, center: MovablePoint)**
- **o MovableRectangle(topLeft: MovablePoint, bottomRight: MovablePoint) (ensuring both points have same speed)**

• Provide toString() to display positions.

• In main(), create a few objects and call move methods to simulate motion.

```
package INterface_practice;
interface Movable{
        void moveUp();
  void moveDown();
  void moveLeft();
  void moveRight();
}
class MovablePoint implements Movable{
        int x, y, xSpeed, ySpeed;
        MovablePoint(int x, int y, int xSpeed, int ySpeed){
                this.x=x;
                this.y=y;
                this.xSpeed = xSpeed;
    this.ySpeed = ySpeed;
        }

        public void moveUp()
        {
                y = y-ySpeed;
        }

        public void moveDown()
        {
                y =y+ ySpeed;
        }

        public void moveLeft()
        {
                x =x-xSpeed;
        }

        public void moveRight()
        {
                x =x+xSpeed;
        }

        public String toString()
        {
```

```
                return "(" + x + ", " + y + ") speed=" + xSpeed + "," + ySpeed;
        }

}
class MovableCircle implements Movable {
        private int radius;
  private MovablePoint center;

  public MovableCircle(int x, int y, int xSpeed, int ySpeed, int radius) {
    this.center = new MovablePoint(x, y, xSpeed, ySpeed);
    this.radius = radius;
  }

  public void moveUp()
  {
        center.moveUp();
  }
  public void moveDown()
  {
        center.moveDown();
  }
  public void moveLeft()
  {
        center.moveLeft();
  }
  public void moveRight()
  {
        center.moveRight();
  }
  public String toString() {
    return "Circle at " + center.toString() + " radius=" + radius;
  }
}
class MovableRectangle implements Movable {
        private MovablePoint topLeft;
  private MovablePoint bottomRight;

  public MovableRectangle(int x1, int y1, int x2, int y2, int xSpeed, int ySpeed) {
    this.topLeft = new MovablePoint(x1, y1, xSpeed, ySpeed);
    this.bottomRight = new MovablePoint(x2, y2, xSpeed, ySpeed);
  }

  public void moveUp() {
    topLeft.moveUp();
    bottomRight.moveUp();
  }
  public void moveDown() {
    topLeft.moveDown();
    bottomRight.moveDown();
```

```
  }
  public void moveLeft() {
     topLeft.moveLeft();
     bottomRight.moveLeft();
  }
  public void moveRight() {
     topLeft.moveRight();
     bottomRight.moveRight();
  }
  public String toString() {
     return "Rectangle [TopLeft: " + topLeft + ", BottomRight: " + bottomRight + "]";
  }
}
public class Shape_simulation {
        public static void main(String[] args) {
                // TODO Auto-generated method stub

                Movable m1 = new MovablePoint(5, 5, 2, 2);
     System.out.println("Initial Point: " + m1);
     m1.moveRight();
     System.out.println("After moving right: " + m1);
     Movable m2 = new MovableCircle(10, 10, 1, 1, 20);
     System.out.println("\nInitial Circle: " + m2);
     m2.moveDown();
     System.out.println("After moving down: " + m2);
     Movable m3 = new MovableRectangle(0, 0, 10, 10, 5, 5);
     System.out.println("\nInitial Rect: " + m3);
     m3.moveUp();
     System.out.println("After moving up: " + m3);
        }
}
```

```
Initial Point: (5, 5) speed=2,2
After moving right: (7, 5) speed=2,2
Initial Circle: Circle at (10, 10) speed=1,1 radius=20
After moving down: Circle at (10, 11) speed=1,1 radius=20
Initial Rect: Rectangle [TopLeft: (0, 0) speed=5,5, BottomRight: (10, 10) speed=5,5]
After moving up: Rectangle [TopLeft: (0, -5) speed=5,5, BottomRight: (10, 5) speed=5,5]
```

2. Default and Static Methods in Interfaces

- **Declare interface Polygon with:**
  - **o double getArea()**
  - **o default method default double getPerimeter(int... sides) that computes sum of sides**
  - **o a static helper static String shapeInfo() returning a description string**
- **Implement classes Rectangle and Triangle, providing appropriate getArea().**
- **In Main, call getPerimeter(...) and Polygon.shapeInfo().**

```
package INterface_practice;
```

```
interface Polygon
{
	double getArea();

	default double getPerimeter(int... sides)
	{
		double sum=0;
		for (int side : sides) {
	    sum = sum + side;
	  }
	  return sum;
	  }
	  static String shapeInfo() {
		return "Polygons";
	  }
}
class Rectangle implements Polygon {
  double length, width;
  Rectangle(double l, double w) {
    this.length = l;
    this.width = w;
  }
  public double getArea() {
    return length * width;
  }
}
class Triangle implements Polygon {
  double base, height;
  Triangle(double b, double h) {
    this.base = b;
    this.height = h;
  }
  public double getArea() {
    return 0.5 * base * height;
  }
}
public class Default_static_method {
	public static void main(String[] args) {
		// TODO Auto-generated method stub

		System.out.println(Polygon.shapeInfo());
    Rectangle rect = new Rectangle(10, 5);

    System.out.println("Rectangle Area: " + rect.getArea());
    System.out.println("Rectangle Perimeter: " + rect.getPerimeter(10, 5, 10, 5));
    Triangle tri = new Triangle(4, 3);
    System.out.println("\nTriangle Area: " + tri.getArea());
    System.out.println("Triangle Perimeter: " + tri.getPerimeter(10, 12, 15));
	  }
```

```
}
Polygons
Rectangle Area: 50.0
Rectangle Perimeter: 30.0
Triangle Area: 6.0
Triangle Perimeter: 37.0
```